



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА

«MANOUT. СИСТЕМА РЕКОМЕНДАЦИЙ»

по дисциплине: «Технологии обработки больших данных»

Выполнил: студент группы ИУК4-72Б

(Подпись)

Моряков В.Ю.

(И.О. Фамилия)

Проверил:

(Подпись)

Голубева С.Е.

(И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Калуга, 2025

Цель: формирование практических навыков работы с библиотекой Mahout для создания рекомендательных систем на основе больших данных.

Задачи:

1. Изучить алгоритмы системы рекомендаций на основе коллаборативной фильтрации.
2. Научиться реализовывать системы рекомендаций с помощью Apache Mahout.
3. Научиться выполнять оценку правильности работы системы рекомендаций.

Вариант №17

Задание:

Реализовать 2 системы рекомендаций фильмов для пользователя на основе его оценок. Использовать реализации рекомендательных систем и метрик, указанные в варианте задания. Сравнить оценки правильности работы всех систем. Для сравнения запускать алгоритм оценки как минимум 10 раз и использовать среднее значение оценки для каждой из систем.

TreeClusteringRecommender с метрикой LogLikelihoodSimilarity;
SVDDRecommender

Смысл алгоритмов:

- 1) TreeClusteringRecommender — это **кластеризационный** рекомендатель. Он строит **иерархическое дерево кластеров пользователей или элементов**, чтобы объединить похожие объекты (например, пользователей с похожими вкусами) и затем использовать это для рекомендаций.
- 2) SVDDRecommender — это **модельный подход** из класса **матриц факторизации**. Он рассматривает данные (пользователь × элемент × рейтинг) как большую разреженную матрицу и пытается аппроксимировать её с помощью скрытых факторов.
 - U — матрица пользователей (их скрытые предпочтения),
 - Σ — диагональная матрица значимости факторов
 - V — матрица элементов (их скрытые характеристики).

Запуск и результаты выполнения:

```
#: mvn clean compile
```

```
#: mvn exec:java -Dexec.mainClass="com.example.recommender.MovieRecommender"
```

```
Data loaded: 10 users, 725 movies
```

```
=====
RECOMMENDATION SYSTEMS COMPARISON
=====
```

```
--- DEMO RECOMMENDATIONS ---
```

```
Recommendations for user 1:
```

```
User-Based (as TreeClustering replacement):
```

```
Movie 1079 (score: 5.000)
```

```
Movie 4034 (score: 5.000)
```

```
Movie 3081 (score: 5.000)
```

```
SVD:
```

Movie 34405 (score: 4.656)
Movie 7153 (score: 4.656)
Movie 44665 (score: 4.656)

Recommendations for user 2:

User-Based (as TreeClustering replacement):

Movie 1198 (score: 5.000)
Movie 165 (score: 5.000)
Movie 1101 (score: 5.000)

SVD:

Movie 1196 (score: 7.140)
Movie 6539 (score: 7.088)
Movie 44665 (score: 6.727)

--- SYSTEM EVALUATION (10 runs) ---

Evaluating: User-Based (TreeClustering replacement)

Run 1: MAE=1.1148, RMSE=1.5421
Run 2: MAE=1.5042, RMSE=1.5910
Run 3: MAE=1.2300, RMSE=1.3411
Run 4: MAE=1.3379, RMSE=1.5140
Run 5: MAE=1.3947, RMSE=1.8180
Run 6: MAE=1.6253, RMSE=1.8025
Run 7: MAE=1.3150, RMSE=1.3861
Run 8: MAE=1.1817, RMSE=1.2760
Run 9: MAE=1.1857, RMSE=1.9413
Run 10: MAE=1.3928, RMSE=1.3138

Evaluating: SVDRecommender

Run 1: MAE=1.4302, RMSE=2.0727
Run 2: MAE=1.6866, RMSE=1.8062
Run 3: MAE=1.0888, RMSE=1.6491
Run 4: MAE=1.7542, RMSE=2.2735
Run 5: MAE=1.4909, RMSE=1.5856
Run 6: MAE=1.0974, RMSE=1.8043
Run 7: MAE=1.2047, RMSE=1.6824
Run 8: MAE=1.2853, RMSE=1.9394
Run 9: MAE=1.2532, RMSE=1.3459
Run 10: MAE=0.9320, RMSE=1.6572

FINAL RESULTS (lower = better):

User-Based (TreeClustering replacement): MAE=1.3282, RMSE=1.5526
SVDRecommender : MAE=1.3223, RMSE=1.7816

=====
BEST SYSTEMS:

=====
Best MAE: SVDRecommender (1.3223)
Best RMSE: User-Based (TreeClustering replacement) (1.5526)

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 48.108 s
[INFO] Finished at: 2025-11-04T10:18:38+00:00
[INFO] Final Memory: 19M/1999M
[INFO] -----

Листинг:

MovieRecommender.java

```
package com.example.recommender;

import org.apache.mahout.cf.taste.common.TasteException;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.NearestNUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
import org.apache.mahout.cf.taste.impl.recommender.svd.ALSWRFactorizer;
import org.apache.mahout.cf.taste.impl.recommender.svd.SVDRecommender;
import org.apache.mahout.cf.taste.impl.similarity.PearsonCorrelationSimilarity;
import org.apache.mahout.cf.taste.model.DataModel;
import org.apache.mahout.cf.taste.neighborhood.UserNeighborhood;
import org.apache.mahout.cf.taste.recommender.RecommendedItem;
import org.apache.mahout.cf.taste.recommender.Recommender;
import org.apache.mahout.cf.taste.similarity.UserSimilarity;
import org.apache.mahout.cf.taste.eval.RecommenderBuilder;
import org.apache.mahout.cf.taste.eval.RecommenderEvaluator;
import org.apache.mahout.cf.taste.impl.eval.AverageAbsoluteDifferenceRecommenderEvaluator;
import org.apache.mahout.cf.taste.impl.eval.RMSRecommenderEvaluator;

import java.io.*;
import java.util.List;
import java.util.ArrayList;

public class MovieRecommender {

    public static void main(String[] args) throws Exception {
        System.out.println("=== MovieLens Recommendation System ===");

        String inputFile = "/datasets/ml-32m/ratings.csv"; // путь к исходному файлу
        String outputFile = "ratings_mahout_1000.csv";

        prepareDataFirst1000(inputFile, outputFile);

        DataModel model = new FileDataModel(new File(outputFile));
        System.out.println("Data loaded: " + model.getNumUsers() + " users, " + model.getNumItems() + " movies");

        compareSystems(model);
    }

    // --- Подготовка данных ---
    static void prepareDataFirst1000(String inputPath, String outputPath) throws IOException {
        List<String> lines = readAllLines(inputPath);
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(outputPath))) {
            int start = lines.get(0).startsWith("userId") ? 1 : 0;
            int count = 0;
            int maxLines = 1000;
            for (int i = start; i < lines.size() && count < maxLines; i++) {
                String[] parts = lines.get(i).split(",");
                if (parts.length >= 3) {
                    writer.write(parts[0] + "," + parts[1] + "," + parts[2] + "\n");
                    count++;
                }
            }
            System.out.println("Processed " + count + " lines (first 1000 rows)");
        }
        System.out.println("Data prepared: " + outputPath);
    }
}
```

```

static List<String> readAllLines(String filePath) throws IOException {
    List<String> lines = new ArrayList<>();
    try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
        String line;
        while ((line = reader.readLine()) != null) {
            lines.add(line);
        }
    }
    return lines;
}

// --- Сравнение двух систем ---
static void compareSystems(DataModel model) throws TasteException {
    System.out.println("\n=====");
    System.out.println(" RECOMMENDATION SYSTEMS COMPARISON ");
    System.out.println("=====");

    showDemoRecommendations(model);
    evaluateSystems(model);
}

static void showDemoRecommendations(DataModel model) throws TasteException {
    System.out.println("\n--- DEMO RECOMMENDATIONS ---");

    UserSimilarity userSimilarity = new PearsonCorrelationSimilarity(model);
    UserNeighborhood neighborhood = new NearestNUserNeighborhood(10, userSimilarity, model);
    Recommender clusteringRecommender = new GenericUserBasedRecommender(model, neighborhood, userSimilarity);

    Recommender svdRecommender = new SVDRecommender(model, new ALSWRFactorizer(model, 10, 0.065, 10));

    int[] testUsers = findAvailableUsers(model);
    for (int testUser : testUsers) {
        System.out.println("\nRecommendations for user " + testUser + ":");

        System.out.println("User-Based (as TreeClustering replacement):");
        printRecommendations(clusteringRecommender.recommend(testUser, 3));

        System.out.println("SVD:");
        printRecommendations(svdRecommender.recommend(testUser, 3));

        if (testUser == testUsers[1]) break; // показываем только двух пользователей
    }
}

static int[] findAvailableUsers(DataModel model) throws TasteException {
    List<Integer> users = new ArrayList<>();
    for (int i = 1; i <= 10; i++) {
        try {
            if (model.getPreferencesFromUser(i).length() >= 2) {
                users.add(i);
            }
            if (users.size() >= 3) break;
        } catch (Exception e) {}
    }
    if (users.isEmpty()) {
        users.add(1);
        users.add(2);
    }
    int[] result = new int[users.size()];

```

```

        for (int i = 0; i < users.size(); i++) result[i] = users.get(i);
        return result;
    }

    static void printRecommendations(List<RecommendedItem> items) {
        if (items.isEmpty()) System.out.println(" No recommendations");
        else for (RecommendedItem item : items)
            System.out.printf(" Movie %d (score: %.3f)\n", item.getItemID(), item.getValue());
    }

    static void evaluateSystems(DataModel model) throws TasteException {
        System.out.println("\n--- SYSTEM EVALUATION (10 runs) ---");

        int runs = 10;
        double[] results1 = evaluateSystem(model, runs, "User-Based (TreeClustering replacement)",
            new RecommenderBuilder() {
                public Recommender buildRecommender(DataModel model) throws TasteException {
                    UserSimilarity similarity = new PearsonCorrelationSimilarity(model);
                    UserNeighborhood neighborhood = new NearestNUserNeighborhood(20, similarity, model);
                    return new GenericUserBasedRecommender(model, neighborhood, similarity);
                }
            });

        double[] results2 = evaluateSystem(model, runs, "SVDRecommender",
            new RecommenderBuilder() {
                public Recommender buildRecommender(DataModel model) throws TasteException {
                    return new SVDRecommender(model, new ALSWRFactorizer(model, 10, 0.065, 10));
                }
            });

        System.out.println("\n-----");
        System.out.println("FINAL RESULTS (lower = better):");
        System.out.println("-----");
        printResults("User-Based (TreeClustering replacement)", results1);
        printResults("SVDRecommender", results2);

        findBestSystem(results1, results2);
    }

    static double[] evaluateSystem(DataModel model, int runs, String name, RecommenderBuilder builder)
        throws TasteException {
        System.out.println("\nEvaluating: " + name);

        RecommenderEvaluator maeEval = new AverageAbsoluteDifferenceRecommenderEvaluator();
        RecommenderEvaluator rmseEval = new RMSRecommenderEvaluator();

        double totalMAE = 0, totalRMSE = 0;
        for (int i = 0; i < runs; i++) {
            try {
                double mae = maeEval.evaluate(builder, null, model, 0.7, 1.0);
                double rmse = rmseEval.evaluate(builder, null, model, 0.7, 1.0);
                totalMAE += mae;
                totalRMSE += rmse;
                System.out.printf(" Run %d: MAE=%.4f, RMSE=%.4f\n", i + 1, mae, rmse);
            } catch (Exception e) {
                System.out.printf(" Run %d: Error in evaluation\n", i + 1);
            }
        }
        return new double[]{totalMAE / runs, totalRMSE / runs};
    }
}

```

```

static void printResults(String name, double[] results) {
    System.out.printf("%-35s: MAE=%.4f, RMSE=%.4f\n", name, results[0], results[1]);
}

static void findBestSystem(double[] res1, double[] res2) {
    String[] names = {"User-Based (TreeClustering replacement)", "SVDRecommender"};
    double[][] all = {res1, res2};

    int bestMAE = res1[0] < res2[0] ? 0 : 1;
    int bestRMSE = res1[1] < res2[1] ? 0 : 1;

    System.out.println("\n=====");
    System.out.println("BEST SYSTEMS:");
    System.out.println("=====");
    System.out.printf("Best MAE: %s (%.4f)\n", names[bestMAE], all[bestMAE][0]);
    System.out.printf("Best RMSE: %s (%.4f)\n", names[bestRMSE], all[bestRMSE][1]);
}
}

```

pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>movie-recommender</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.apache.mahout</groupId>
            <artifactId>mahout-mr</artifactId>
            <version>0.13.0</version>
        </dependency>
        <dependency>
            <groupId>org.apache.mahout</groupId>
            <artifactId>mahout-integration</artifactId>
            <version>0.13.0</version>
        </dependency>
        <dependency>
            <groupId>com.google.guava</groupId>
            <artifactId>guava</artifactId>
            <version>19.0</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>

```

```
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
  <source>1.8</source>
  <target>1.8</target>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

Вывод: в ходе выполнения лабораторной работы были реализованы две рекомендательные системы по подбору рекомендаций фильмов.